

Week 02

머신러닝 기초 학습 자료

- 일시: 2026.09.28
- 참석 인원: 3명 / 전체 3명
- 참석: 성준서, 정예빈, 김영빈

활동 내용

- 갈라파고스 핀치 부리 데이터를 활용한 머신러닝 기초 학습
- 특성, 지도 학습과 비지도 학습, 훈련·테스트 세트, KNN, 정확도, 데이터 전처리 정리
- NumPy와 Matplotlib으로 데이터를 나누고 산점도 시각화
- 변수명 오타, 점의 겹침, 노트북 실행 순서에 관한 궁금증을 AI에게 질문하고 정리
- 실습 화면과 스터디 기록을 GitHub Pages로 공유

학습 내용

기본 개념

1. 특성과 학습

- **특성(Feature)**: 데이터를 표현하는 성질. 이번 노트에서는 부리 폭과 길이를 두 특성으로 다룸.
- **학습(fit)**: 데이터에서 규칙을 찾는 과정. 사이킷런에서는 `fit()`으로 모델을 학습시킴.
- **모델(Model)**: 알고리즘을 구현한 객체. 실습에서는 `knn`이라는 변수로 다룸.

2. 지도 학습과 비지도 학습

구분	학습에 사용하는 정보	목적
지도 학습	입력 데이터와 정답	새로운 데이터의 정답 예측. 이번에 다룬 KNN 분류가 해당됨.
비지도 학습	정답이 없는 입력 데이터	데이터 내부의 구조나 패턴 탐색

3. 훈련 세트와 테스트 세트

- **훈련 세트**: 모델이 학습할 때 사용하는 데이터.
- **테스트 세트**: 학습에 사용하지 않고, 새로운 데이터에 대한 모델의 성능을 평가하는 데이터.
- 원문에서는 전체 데이터의 **20~50%를 테스트 세트로 두는 방식**을 소개함. 분할 비율은 데이터 규모와 평가 목적에 맞게 결정함.
- 훈련 데이터는 충분한 양과 대표성을 확보하는 것이 중요함.

4. k-최근접 이웃(KNN)

- 비교적 이해하기 쉬운 머신러닝 알고리즘 중 하나.
- 훈련 데이터를 저장하고, 예측할 데이터와 가까운 **k개의 이웃**을 찾아 분류함.
- 기본적인 분류 방식에서는 가까운 이웃 중 더 많은 클래스에 속하는 것으로 예측함.
- 이웃 수는 3, 5, 7과 같은 홀수를 사용해 볼 수 있음. 이진 분류에서 같은 수의 표가 나오는 경우를 줄이는 데 도움이 됨.
- 여러 이웃 수에 따른 성능을 비교하며 적절한 값을 탐색함.

5. 정확도

정확도 = 맞힌 개수 / 전체 개수

사이킷런의 분류 정확도는 0~1 범위로 표현됨. 예측값과 정답을 비교한 불리언 배열의 평균으로도 계산할 수 있음.

```
pred = knn.predict(test_input)
accuracy = np.mean(pred == test_target)
print(accuracy)
```

위 코드는 np, 학습된 knn, test_input, test_target이 준비된 상태에서 실행하는 예시임.

6. 데이터 전처리

- **데이터 전처리**: 모델에 데이터를 넣기 전에 가공하는 단계. 전체 작업에서 많은 시간을 차지할 수 있음.
- **표준점수**: 각 값에서 평균을 빼고 표준편차로 나누어 특성의 스케일을 맞춤.
- **표준점수 = (값 - 평균) / 표준편차**
- 테스트 세트에도 **훈련 세트에서 구한 평균과 표준편차**를 적용해야 함.
- **브로드캐스팅**: NumPy 배열의 형태가 호환될 때, 크기가 다른 배열 사이에서도 연산할 수 있도록 하는 기능.

사용 함수와 옵션

라이브러리	함수·옵션	용도
NumPy	np.array()	리스트를 배열로 변환
NumPy	np.arange()	일정한 간격을 갖는 값의 배열 생성
NumPy	np.random.seed()	난수 생성 결과를 재현할 수 있도록 시드 설정
Matplotlib	plt.scatter()	산점도 그리기
Matplotlib	c=	점의 색 지정. 별도 지정이 없으면 기본 색상 순환을 사용하며, 기본 설정은 10색임.
Matplotlib	s=, alpha=	점의 크기와 투명도 조절
scikit-learn	train_test_split()	훈련 세트와 테스트 세트 분리
scikit-learn	stratify=	지정한 클래스 비율을 유지하도록 분할
scikit-learn	KNeighborsClassifier()	KNN 분류 모델 생성
scikit-learn	kneighbors()	가까운 이웃의 거리와 인덱스 확인

실습: 갈라파고스 핀치 부리

실습 화면: 뒤에 첨부한 실습 사진 자료에서 확인.

특성: beak_height는 부리의 폭, beak_length는 부리의 길이임.

- A섬 핀치: 부리의 폭이 작고 길이가 김.
- B섬 핀치: 부리의 폭이 크고 길이가 짧음.

원문 실습에서는 부리 폭과 길이로 구성된 **49마리 데이터**를 다음 두 그룹으로 나눔.

구분	인덱스	개수	슬라이싱	그래프 색상
훈련 세트	0~34번	35마리	[:35]	파란색
테스트 세트	35~48번	14마리	[35:]	주황색

아래 코드는 beak_data와 beak_target에 원본 데이터가 미리 입력되어 있다는 전제의 실습 코드임. 링크된 페이지에는 원본 배열의 전체 값이 텍스트로 제공되어 있지 않음.

```
import numpy as np
import matplotlib.pyplot as plt

beak_data = np.array(beak_data)
beak_target = np.array(beak_target)

train_input = beak_data[:35] # 0~34번
test_input = beak_data[35:] # 35~끝
train_target = beak_target[:35]
test_target = beak_target[35:]

plt.scatter(train_input[:, 0], train_input[:, 1], c="blue") # 훈련
plt.scatter(test_input[:, 0], test_input[:, 1], c="orange") # 테스트
plt.xlabel("beak_length")
plt.ylabel("beak_height")
plt.show()
```

훈련 세트와 테스트 세트 구분 완성본: 뒤에 첨부한 실습 사진 자료에서 확인.

실습 결과에서는 파란 점들이 위쪽에, 주황 점들이 아래쪽에 나뉘어 나타남.

질문 및 정리

Q1. train 철자만 고쳤는데 왜 그래프가 완전히 달라졌을까?

train_input을 trian_input으로 잘못 입력했다가 수정했더니 파란 점 35개가 보이지 않고 주황색만 남는 상황이 발생함.

변수명 철자가 틀렸을 때: 뒤에 첨부한 실습 사진 자료에서 확인.

오타가 있을 때의 상황

- trian_input에 대입하면 기존 train_input이 수정되는 것이 아니라 별개의 변수에 값이 저장됨.
- 노트북 커널에는 이전에 실행한 결과가 남아 있어, 기존 train_input의 옛 값을 계속 사용할 수 있음.
- 원문에서 설명한 당시 상태에서는 파란색으로 옛 데이터 14마리, 주황색으로 새 데이터 35마리가 표시됨.

trian_input에 대입한 상태
파란색: 기존 train_input에 남은 아래쪽 14마리
주황색: test_input에 들어간 위쪽 35마리
→ 서로 다른 위치이므로 두 색이 모두 보임

철자를 고쳤을 때의 상황

변수명 철자를 고쳤을 때: 뒤에 첨부한 실습 사진 자료에서 확인.

```
train_input = beak_data[:35] # 수정 후 35마리로 갱신
test_input = beak_data[:35] # 잘못된 분할: 같은 35마리를 선택
```

- 두 변수에 같은 35마리 데이터가 들어가 동일한 좌표에 점을 찍게 됨.
- 나중에 그린 주황색 점이 파란색 점을 덮어 가림.
- 파란 점이 삭제된 것이 아니라 **두 데이터가 겹쳐 보인 것임**.

해결 방법: 테스트 데이터의 슬라이싱을 [35:]로 수정함.

```
train_input = beak_data[:35]
test_input = beak_data[35:]
```

변수명뿐 아니라 두 줄에 같은 [:35]를 사용하고 있는지도 함께 확인해야 함.

Q2. 파란 점이 14개인데 왜 8개 정도로만 보일까?

- 데이터가 사라진 것이 아니라, 좌표가 가까워 여러 점이 겹쳐 보이는 현상임.
- `scatter()`는 각 좌표에 점을 그리므로 가까운 점들이 서로 가려질 수 있음.
- 원문 기록에서는 49개 행을 비교했을 때 완전히 동일한 행은 없었고, **0.1~0.2 정도의 작은 차이** 때문에 점들이 가까이 표시된 것으로 설명함.
- 점의 크기와 투명도를 조정하면 겹침을 관찰하는 데 도움이 됨.

```
plt.scatter(x, y, s=60, alpha=0.6)
```

x, y에는 확인하려는 두 특성의 데이터를 넣음.

Q3. 노트북은 왜 위에서 아래로 실행되는 것처럼 보이지 않을까?

같은 변수명을 여러 셀에서 반복해서 사용하면서, 아래쪽 셀의 실행 결과가 위쪽 셀을 다시 실행할 때 영향을 주는 상황을 경험함.

핵심은 **노트북에 적힌 순서와 실제로 셀을 실행한 순서가 다를 수 있다**는 점임.

구분	.py 파일을 일반적으로 실행할 때	Jupyter / Colab 노트북
실행 방식	프로그램의 제어 흐름에 따라 실행	원하는 셀을 선택해 실행 가능
변수 상태	실행 중인 프로세스 안에서 유지되며, 프로세스 종료 시 사라짐	커널 런타임이 유지되는 동안 이전 실행 결과가 남음
다시 실행할 때	파일을 새 프로세스로 실행하면 처음부터 실행	선택한 셀만 실행되며, 다른 셀의 출력은 자동으로 갱신되지 않음

※ 원문의 비교 취지를 유지하면서, .py 재실행 방식과 노트북 변수의 유지 기간은 위와 같이 표현을 보완함.

- 같은 변수명을 여러 번 사용하면 현재 값이 어느 셀의 실행 결과인지 추적하기 어려워짐.
- 저장하거나 공유하기 전에는 **커널 런타임을 재시작하고 위에서부터 모두 실행**하여, 이전 실행 상태에 의존하지 않는지 확인함.

실습 메모

1. 같은 변수명을 재사용하면 마지막으로 실행한 셀의 값이 남아 결과에 영향을 줄 수 있음.
2. 아래쪽 셀에서 바꾼 변수는 위쪽 셀을 다시 실행할 때도 사용됨.
3. 변수 재사용으로 의도하지 않은 값이 사용되지 않도록 실행 순서와 현재 상태를 확인해야 함.

스터디 현장

실습 중: 뒤에 첨부한 실습 사진 자료에서 확인.

열심히 하는 우리들: 뒤에 첨부한 실습 사진 자료에서 확인.

한 줄 회고

변수를 단순히 데이터를 담는 상자로 생각했지만, 실제로는 **메모리의 객체를 가리키는 이름**이라는 점을 실습으로 체감함. 변수명 오타 하나가 그래프 전체를 바꾼 경험을 통해, 각 변수에 어떤 값이 연결되어 있고 어떤 순서로 실행했는지 확인하는 습관의 필요성을 느낌.

자료 안내

교재: 혼자 공부하는 머신러닝+딥러닝

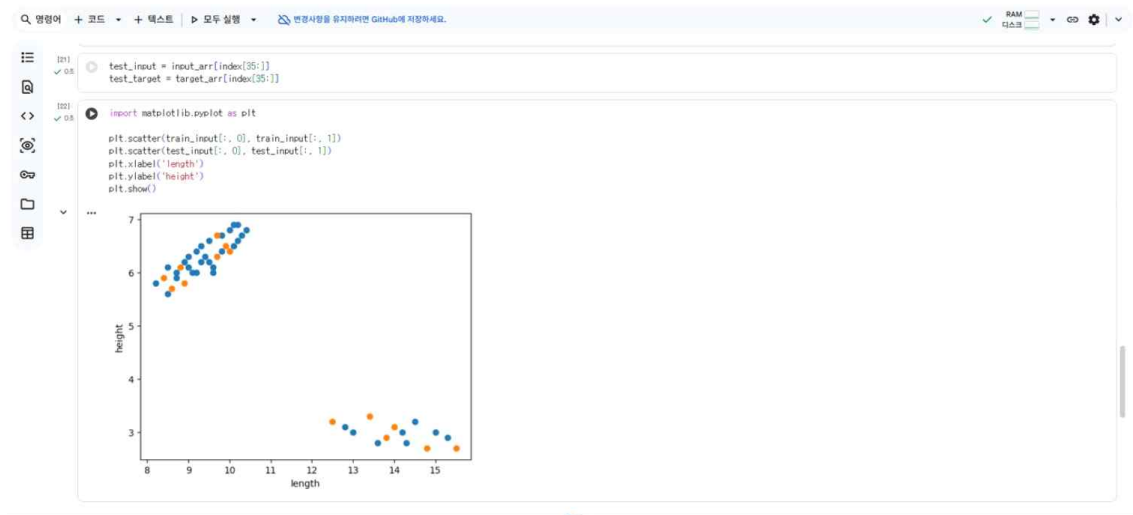
이 PDF는 학습 설명과 실습 코드를 정리한 본문에, 제공받은 실습.pdf 4쪽을 부록으로 합친 제출 자료임. 부록에는 실습 화면 4장과 스터디 사진 2장이 포함됨. 원본 데이터 배열은 포함되어 있지 않아 코드 실행에는 별도 데이터가 필요함.

기록 출처: <https://rlaguswl25808134-source.github.io/AID-MLB/ml-week2.html>

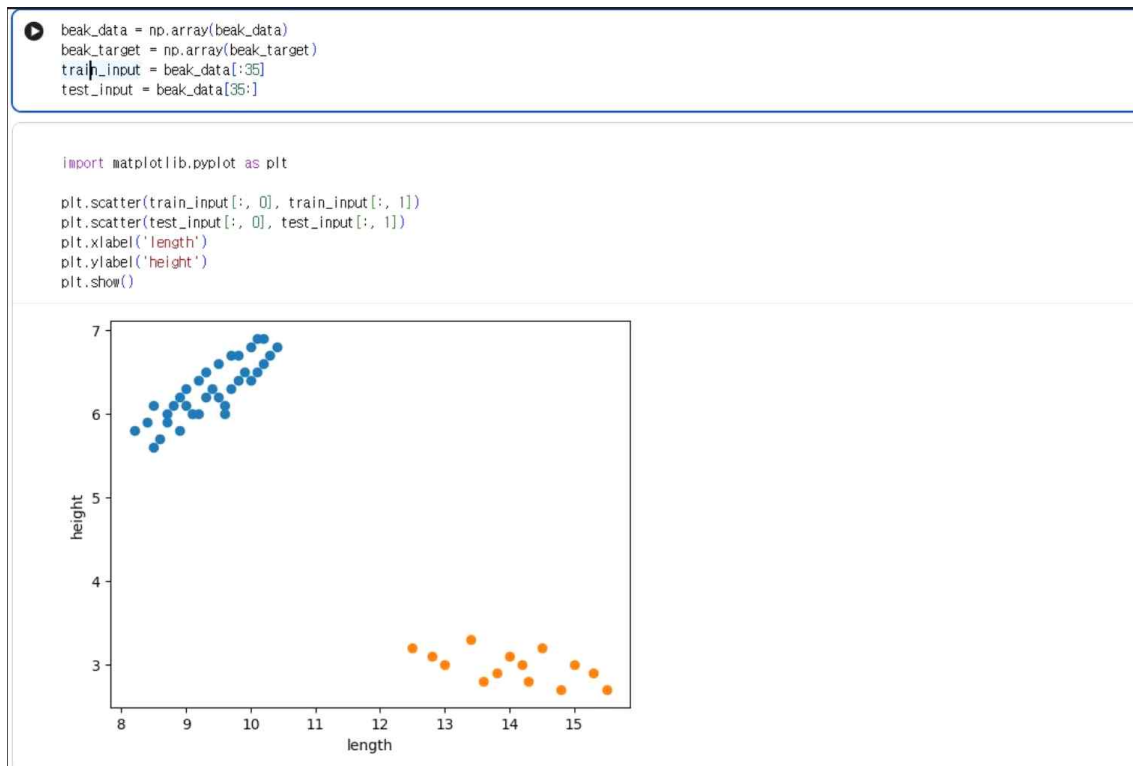
다음 주 계획

훈련 세트와 테스트 세트, 샘플링 편향, 데이터 전처리, 표준점수 학습.

실습: 갈라파고스 핀치 부리



[훈련 세트와 테스트 세트 구분 완성본]



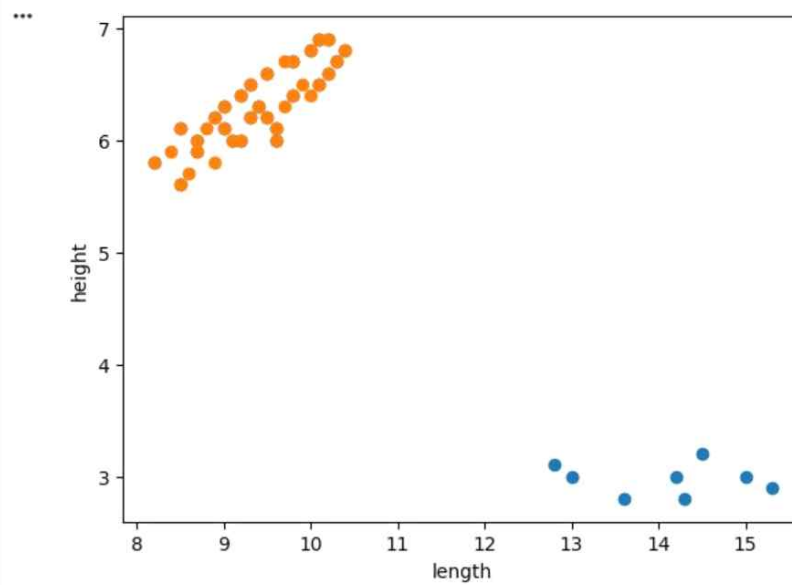
[변수명 철자가 틀렸을 때]

```
beak_data = np.array(beak_data)
beak_target = np.array(beak_target)
trian_input = beak_data[:35]
test_input = beak_data[35]
```

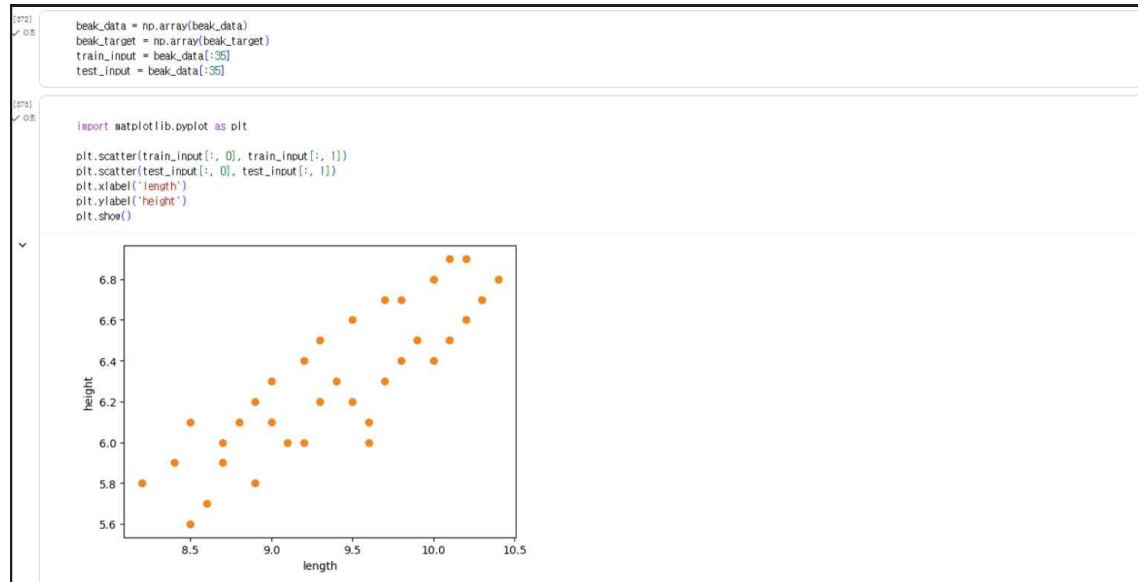


```
import matplotlib.pyplot as plt

plt.scatter(train_input[:, 0], train_input[:, 1])
plt.scatter(test_input[:, 0], test_input[:, 1])
plt.xlabel('length')
plt.ylabel('height')
plt.show()
```

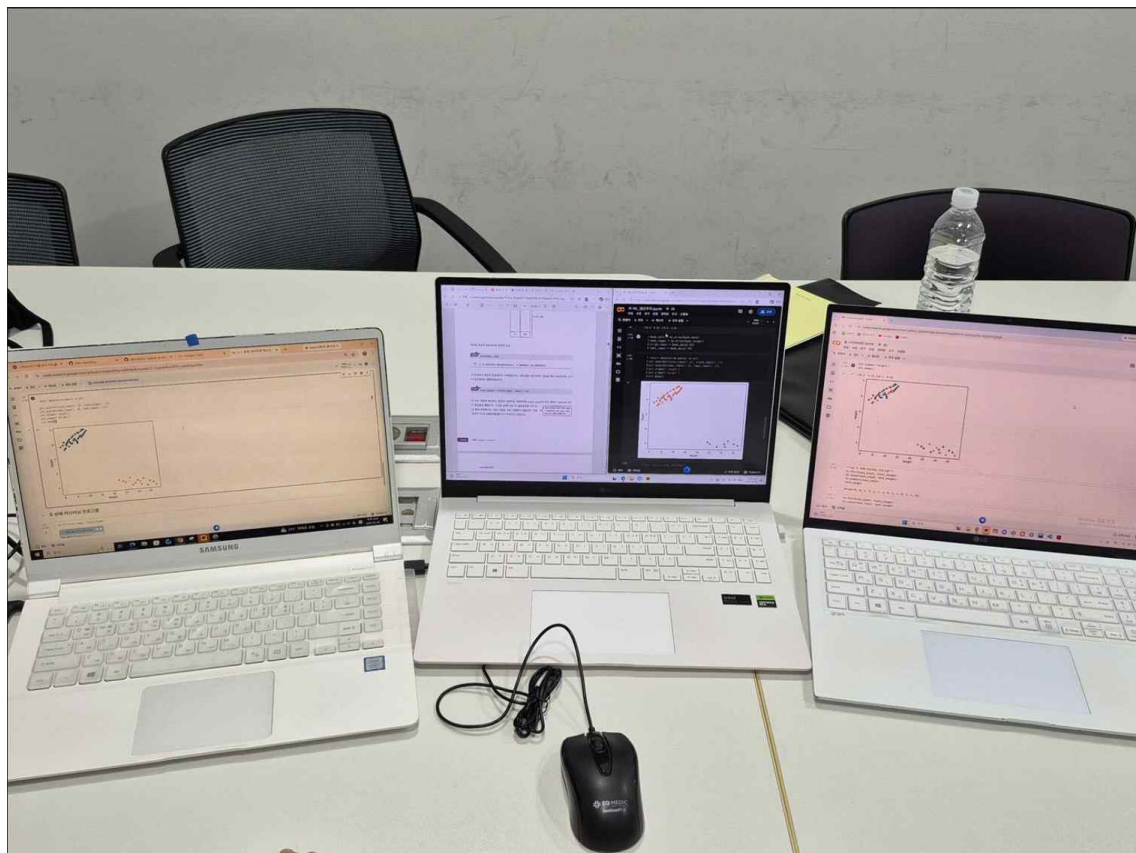


[변수명 철자를 고쳤을 때]



스터디 현장

[실습 중]



[열심히 하는 우리들]

